| | | |
|---|---|---|

**CSE222 / BİL505**

**Data Structures and Algorithms**

**Homework #6 – Report**

**MUHAMMET FURKAN YILGIN**

## 1) Selection Sort

| | |
|---|---|
| **Time Analysis** | Average,worst and best case time complexity of selection sort is the same and it is $O(n^2)$.The reason it has a quadratic time complexity is because it uses a for loop that iterates until the length of the array - 1 which is n-1 and in the inner for loop it iterates from the index in the outer loop+1 to the last element in the array.Therefore the inner loop has a complexity from n to 1 in order to find the minimum element and swap with the current index in the outer loop.It results in [n.(n-1) / 2] complexity this is why it has a quadratic time complexity. |
| **Space Analysis** | Space complexity of selection sort is O(1).The reason of its space complexity being constant is because selection sort does not use any extra array or recursion to swap or store its data and it only uses its own array to do the sorting so that is why it has a constant O(1) space complexity same as bubble sort. |

## 2) Bubble Sort

| | |
|---|---|
| **Time Analysis** | Average and worst case time complexity of bubble sort is $O(n^2)$. However for the best case it has a time complexity of O(n).In the worst case the loop never breaks and continues till the n-1 of the outer loop from index i but in the average case there is a possibility that in the inner loop if there has been no swaps occured it means that the sort is complete before reaching to the (n-1)element in the outer loop however it still ends in quadratic time since the inner loop is iterated multiple times.Finally in best case it only iterates 1 time in outer loop and n-1 times in the inner loop and stops since the array is already sorted and no swaps made resulting in n-1 which is O(n) time complexity. |
| **Space Analysis** | Space complexity of bubble sort is O(1).The reason of its space complexity being constant is because bubble sort does not use any extra array or recursion to swap or store its data and it only uses its own array to do the sorting so that is why it has a constant O(1) space complexity same as selection sort. |

## 3) Quick Sort

| | |
|---|---|
| **Time Analysis** | Average and best case time complexity of quick sort is O(nlogn). However the worst case time complexity of it is $O(n^2)$.For the worst case the pivot selection results in unbalanced partitions where the array is only partioned by one element at a time which results in $O(n^2)$ time complexity.However, for best and average cases the pivot selection results in reasonably balanced partitions and this partioning process takes O(n) time because of looking for every element in the array..For recursion since the pivot divides the array roughly in half it results in logn recursion calls therefore from logn recursive calls and n time for partition it has a time complexity of O(nlogn) for best and average cases. |
| **Space Analysis** | Space complexity of quick sort is O(logn) for best and average cases.It is because it uses recursion to split the array according to the pivot and in both cases pivot selected nearly or exactly at half point resulting in balanced partition and therefore it will have a balanced recursion tree with a call stack of size O(log n).However for the worst case as explained |

| | | |
|---|---|---|

| | |
|---|---|
| | above the partioning will be unbalanced where the pivot will be the leftmost element generally therefore the recursion depth can be equal to the size of the array resulting in O(n) space complexity. |

### 4) Merge Sort

| | |
|---|---|
| **Time Analysis** | Average, worst and best case time complexity of merge sort is the same and it is O(nlogn).In merge sort, the array is divided in half until each sub-array contains only one element then these sub-arrays are merged in sorted order in a divide and conquer approach and this merge operation takes O(n) time and the main array is divided into logn sub-arrays recursively therefore it has a time complexity of O(nlogn) regardless of the cases. |
| **Space Analysis** | Space complexity of merge sort is O(n).It is because it uses an extra array with the same length of the main array to store the merged array and recursion therefore it is O(n). |

## General Comparison of the Algorithms

First of all I would like to compare the space complexities of these 4 sorting algorithms.As it could be seen from the table above in terms of space complexity selection and bubble sort is way better than merge and quick sort.The reason of this comes from neither both selection and bubble sorts use any extra array to store its data nor rely on recursion calls and these reasons result in O(1) constant space usage for both algorithms.However merge sort uses extra array to store its data with recursion and quick sort uses recursion to find the pivot these effects make them use extra space and make their space complexity much more compared to bubble and selection sort.

In terms of time complexity, despite of having their space complexity much greater compared to bubble sort and selection sort , both merge sort and quick sort is so much better for a faster sorting.However , time complexity of quick sort and bubble sort change according to the worst case which is a reversely ordered array , average case and best case where the array is in sorted order while merge and bubble sort has the same time complexity for all 3 cases.For the worst case among these 4 sorting algorithms merge sort is the best one as it can be seen from the table above with the time complexity O(nlogn) and it can also be seen from our example in the homework with the comparison counter being so much low compared to other algorithms because of using a balanced recursion tree with merging.For the average case both merge and quick sort is fast with time complexity of O(nlogn) however as it can be seen from the example in our homework comparison counter of quick sort is lower than merge sort it means in some cases quick sort might be slightly faster than merge sort and also quick sort uses less space with O(logn) for the average case.Finally for the best case bubble sort is the best because of having a time complexity of O(n) as it can be seen from the example in our homework of having the less comparison counter number among other algorithms with O(nlogn) and $O(n^2)$ and no swaps.As explained in tables above.

Finally, swapping does not affect time or space complexity much since they generally have O(1) time and space complexity and involve only a few memory operations therefore comparing the swap counters of the sorting algorithms is not as useful compared to the comparison counters.

| | | |
|---|---|---|